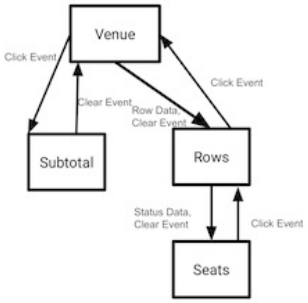
rows, the situation gets even more complicated. The click event on a Seat would need to be passed all the way back up to the Venue and then down to the subtotal calculator. Similarly, a click on the “Clear All” button would need to be passed up to the Venue and down to all the rows to clear all of the user’s seats.

Here’s a diagram:



This promises to be a huge pain—and is exactly the kind of thing that has caused me to avoid React sometimes. But the combination of contexts and reducers allows us to avoid much of this rigmarole. We can store the reducer inside the context and get safe global data.

The context allows us to set the initial state globally when the page loads, and then each individual Seat component can query the context to determine its own status without having to deal with passing props around. Using a reducer inside the context guarantees that the global data will remain consistent. The result is that not only do we have less data being passed through the system but display logic can also end up situated closer to the actual display element. In this case, we specifically wind up moving a lot of display code from the Row component to the Seat component.

The code looks more like this: